

CZBASIC

Graphics Extensions Reference

The CZ statements and functions that CZBASIC 1.0 adds to Microsoft BASIC-80 (MBASIC) 5.21 for the CADzilla S-100 graphics board: Hitachi HD63484 ACRTC and Brooktree Bt453 video DAC

Document revision 1.0 · 2026-09-29

Contents

1. Introduction 3

1.1 Starting CZBASIC . . . 3

1.2 A first program . . . 3

2. Concepts 4

2.1 The screen and coordinates . . . 4

2.2 Arguments . . . 4

2.3 Colors and the palette . . . 4

2.4 The pen . . . 5

2.5 Drawing modes . . . 5

2.6 Patterns and line styles . . . 6

2.7 Drawing and BASIC run side by side . . . 6

2.8 Errors . . . 6

3. Statements 7

3.1 Setting up . . . 7

3.2 Color and palette . . . 8

3.3 Modes and the area . . . 8

3.4 The pen . . . 9

3.5 Dots and lines . . . 10

3.6 Rectangles . . . 11

3.7 Circles, ellipses and arcs . . . 11

3.8 Polygons and polylines . . . 13

3.9 Filling areas . . . 13

3.10 Copying . . . 14

3.11 Patterns . . . 14

3.12 Text . . . 16

4. Functions 18

5. Example programs 19

5.1 A star burst . . . 19

5.2 A rubber band with XOR . . . 19

5.3 Shapes, paint and flood . . . 19

5.4 Reading the screen . . . 19

5.5 Palette animation . . . 19

Appendix A. Quick reference 20

1. Introduction

CZBASIC runs programs written for Microsoft BASIC-80 (MBASIC) 5.21 under CP/M, and adds statements and functions for drawing on the CADzilla board. Their names all begin with **CZ**. This manual documents only those extensions. For the rest of the language — PRINT, FOR, GOSUB, files, EDIT and so on — see Microsoft's *BASIC-80 Reference Manual* for version 5.21: CZBASIC follows it.

1.1 Starting CZBASIC

```
A0>CZBASIC                start at command level (Ok)
A0>CZBASIC CZDEMO         load and run CZDEMO.BAS
A0>CZBASIC BIGPROG /X     no graphics: more memory for programs
```

CZBASIC accepts MBASIC's switches (`/F:` files, `/M:` highest address, `/S:` record size) and one of its own, `/X`. With /X the memory that holds the graphics code (3,743 bytes) is given to programs instead, for large programs that don't draw; any CZ statement or function then gives *Illegal function call*.

1.2 A first program

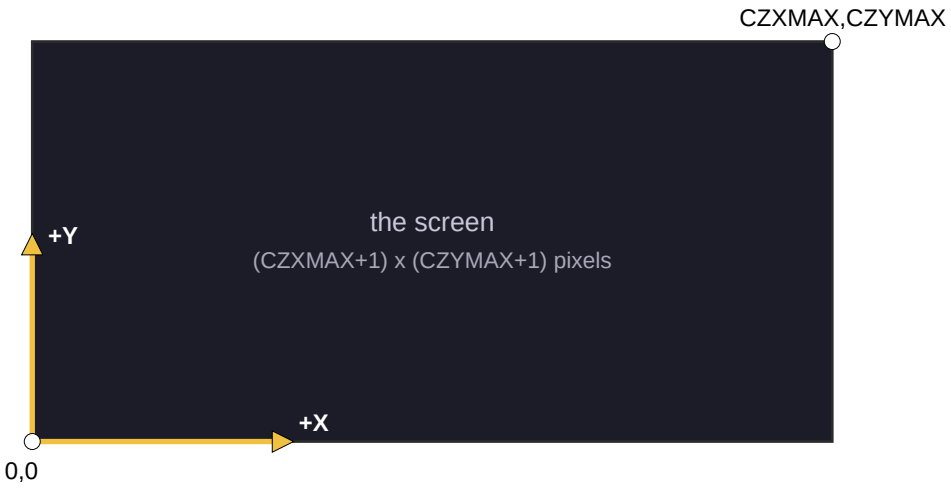
```
10 CZINIT                         ' set up the board, clear the screen
20 CZRECT 0,0,CZXMAX,CZYMAX,255   ' a white frame round the screen
30 FOR R=20 TO 300 STEP 20
40   CZCIRCLE CZXMAX\2,CZYMAX\2,R,64+R\12   ' green circles
50 NEXT
60 CZTEXT 20,CZYMAX-40,"HELLO, CADZILLA",192+31,3
70 CZWAIT
```

Every graphics program starts with **CZINIT**, which programs the board for the monitor, loads the default palette and clears the screen. The drawing statements then take pixel coordinates and an optional color. The sample program `CZDEMO.BAS` tours most of them.

2. Concepts

2.1 The screen and coordinates

Coordinates are pixels. **0,0 is the bottom-left pixel**; X grows to the right and **Y grows upward**, as on graph paper (not downward as on many other computers). The largest coordinates are the functions **CZXMAX** and **CZYMAX**, which depend on the mode CZINIT set:



CZINIT	Pixels	CZXMAX	CZYMAX
CZINIT or CZINIT 1024	1024 x 768	1023	767
CZINIT 800	800 x 600	799	599
CZINIT 640	640 x 480	639	479

The mode must match the monitor the board is set up for (the project's machine is strapped for 1024 x 768 unless it says otherwise). Write programs in terms of CZXMAX and CZYMAX and they work in every mode.

Coordinates may be negative or beyond the screen; a shape can lie partly off it. Nothing is clipped unless you ask for it, though, and the parts off the screen can land elsewhere in video memory, even reappearing on the screen. To keep drawing inside the screen, turn on clipping with an area mode: `CZMODE 0,0,2` (see 2.5 and CZAREA).

2.2 Arguments

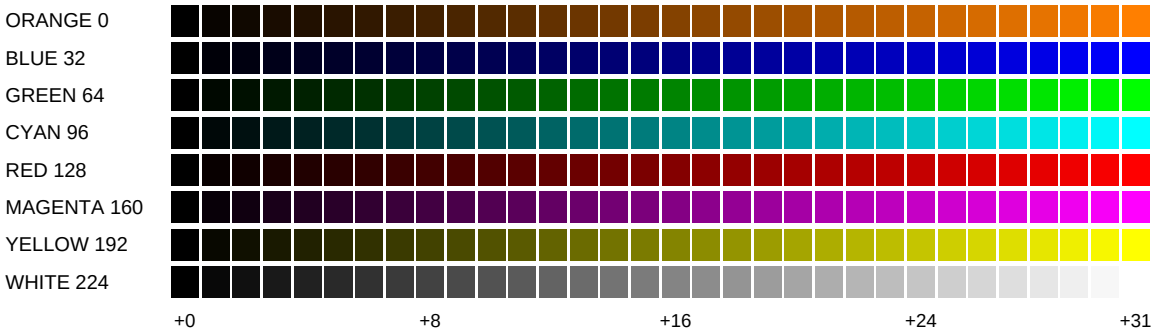
Every argument of a CZ statement is a numeric expression: a constant, a variable, or any formula. It is rounded to a whole number the way CINT rounds (1.5 becomes 2, -1.5 becomes -2) and must lie in -32768 to 32767, else *Overflow*. Arguments shown in square brackets in the syntax may be left off, together with every argument after them. Colors, palette values and similar small numbers use the low 8 bits of the value (0 to 255).

Plain constants and simple variables as arguments are the fastest; a formula works just as well but has to be worked out first.

2.3 Colors and the palette

The board shows 256 colors at once. A color is a **palette index**, 0 to 255, and the Bt453 palette maps each index to one of 16.7 million shades (CZPAL changes it). CZINIT loads the **default palette**: eight hues of 32

brightness levels, **index = hue + level**, level 0 being black and 31 the brightest:



So 224+31 = 255 is bright white, 128+31 = 159 bright red and 32+16 = 48 a medium blue. Index 0 (and every level 0) is black.

The **drawing color** is set by **CZCOLOR** and stays until changed; CZINIT makes it 255. Most drawing statements also take an optional color as their last argument, which is used for that statement only:

```
CZCOLOR 128+31                  ' red from now on
CZLINE 0,0,100,100              ' red
CZLINE 0,100,100,0,32+31        ' blue, this line only
CZLINE 50,0,50,100              ' red again
```

CZCOLOR also sets the **background color** (CL0), used by patterns and line styles for their 0 bits (see 2.6); CZINIT makes it 0, black.

2.4 The pen

The board keeps a current position, the **pen**. CZMOVE and CZRMOVE move it; CZLINE TO and CZRLINE draw from it; most other statements move it as they draw. The functions **CZXPOS** and **CZYPOS** read it back. Each statement's entry says where it leaves the pen. CZINIT puts it at 0,0.

2.5 Drawing modes

CZMODE *op*, *cm*, *am* sets three things that apply to all drawing from then on (CZINIT sets all three to 0):

op	Operation: how the color C being drawn meets the pixel P already there
0	Replace: P becomes C (normal drawing)
1	OR: P becomes P OR C
2	AND: P becomes P AND C
3	Exclusive OR: P becomes P XOR C. Drawing the same shape twice restores the screen: rubber-band lines, blinking cursors.
4	Draw only on pixels of color 0 (the compare color, CCMP, which CZINIT sets to 0)
5	Draw only on pixels that are not color 0
6	Draw only where P < C
7	Draw only where P > C

cm	Color mode: what each bit of the pattern draws (see 2.6)
0	A 1 bit draws the drawing color, a 0 bit the background color

cm	Color mode: what each bit of the pattern draws (see 2.6)
1	A 1 bit draws the drawing color, a 0 bit leaves the screen alone (a transparent pattern)
2	A 0 bit draws the background color, a 1 bit leaves the screen alone
3	The pattern RAM is a 4 x 4 tile of colors: word row*4+column holds a color in both bytes (color*257)

am	Area mode: checked against the rectangle CZAREA sets (the whole screen after CZINIT)
0	No checking
1	Stop drawing at the first pixel outside the area
2	Draw only inside the area (clip)
3	As 2, and note that something was clipped
5	Stop drawing at the first pixel inside the area
6	Draw only outside the area (a hole)
7	As 6, and note that something was left out
4	Not used

After a stop mode has stopped a drawing, CZXPOS and CZYPOS give the pixel where it stopped. The "note" in modes 3 and 7 is a status flag of the ACRTC that BASIC does not read; for BASIC, 3 acts as 2 and 7 as 6.

2.6 Patterns and line styles

The ACRTC draws every pixel through a 16 x 16 bit **pattern RAM**: 16 words, word *n* being row *n* of the pattern and bit *m* its column *m* (bit 0 is the leftmost pixel, row 0 the bottom row of a fill). A pattern **pointer** walks it as the chip draws: one bit per pixel along a line (carrying on round the corners of a rectangle, polygon or circle) and one row per raster in a fill, inside a **scan window**. Each bit then draws as the color mode says.

CZINIT fills the pattern RAM with ones and opens the window to all 16 x 16, so everything is drawn solid in the drawing color. **CZLSTYLE** sets a dashed line style, **CZPAT** loads the pattern RAM, **CZPSCAN** sets the window and zoom, and **CZPTN** draws the pattern RAM itself as a small picture. Line styles and patterns apply to fills too. CZTEXT uses the pattern RAM for its characters and leaves it solid again, so set a pattern or line style again after CZTEXT.

2.7 Drawing and BASIC run side by side

A CZ statement hands its drawing to the ACRTC and BASIC carries on while the chip draws. The chip always finishes one drawing before the next, and CZPOINT, CZXPOS and CZYPOS see the results of all the drawing before them. **CZWAIT** waits until everything has been drawn: use it before timing a drawing, and at the end of a program.

2.8 Errors

Error	When
Syntax error	Too few or too many arguments, a missing comma or parenthesis, or CZPAL or CZAREA with 1 to 3 arguments
Illegal function call	A value out of range (each entry lists its limits), or any CZ keyword when CZBASIC was started with /X

Error	When
Type mismatch	A string where a number is needed, or a CZPOLY / CZPLINE array that is not an integer (%) array
Overflow	An argument outside -32768 to 32767

CZ errors are ordinary BASIC errors: ON ERROR GOTO traps them, and ERR and ERL report them. The CZ keywords are reserved words, so a variable name cannot begin with CZ.

3. Statements

The statements in groups: setting up, color, modes, the pen, lines, shapes, fills, copying, patterns and text. Appendix A lists them all on one page.

3.1 Setting up

CZINIT

```
CZINIT [mode]
```

Sets the board up for a display mode and clears the screen to black. Also loads the default palette and puts everything back to its starting state: drawing color 255, background color 0, CZMODE 0,0,0, the area the whole screen, the pattern RAM solid, the pen at 0,0.

Argument	Meaning
mode	1024 (the default), 800 or 640: the screen width. It must match the monitor the board drives.

Errors: Illegal function call for any other mode.

Example

```
10 CZINIT          ' 1024 x 768
10 CZINIT 640      ' 640 x 480
```

CZCLS

```
CZCLS [c]
```

Clears the whole screen to color c. Nothing else changes.

Argument	Meaning
c	The color, 0-255 (default 0, black)

Example

```
CZCLS             ' black
CZCLS 32+4        ' dark blue
```

CZWAIT

```
CZWAIT
```

Waits until the ACRTC has finished all the drawing given to it. Drawing statements return as soon as the chip has their commands, so use CZWAIT before measuring how long a drawing takes, before reading the board with INP, and at the end of a program.

3.2 Color and palette

CZCOLOR

```
CZCOLOR fg[,bg]
```

Sets the drawing color, which stays until the next CZCOLOR or CZINIT, and optionally the background color that patterns and line styles draw for their 0 bits.

Argument	Meaning
fg	Drawing color, 0-255
bg	Background color, 0-255 (unchanged if left off)

Example

```
CZCOLOR 192+31          ' bright yellow
CZCOLOR 255,32+8        ' white, dark blue background
```

CZPAL

```
CZPAL i,r,g,b

CZPAL
```

Sets palette entry i to the color with red, green and blue r, g, b. Everything already drawn in color i changes at once, which makes palette animation cheap. CZPAL with no arguments loads the default palette again.

Argument	Meaning
i	Palette index, 0-255
r, g, b	Red, green and blue, 0-255 each

Errors: Syntax error with 1, 2 or 3 arguments.

Example

```
CZPAL 1,255,128,0        ' index 1: orange
FOR K=0 TO 255:CZPAL 255,K,K,255:NEXT   ' entry 255: blue to white
CZPAL                    ' the default palette again
```

3.3 Modes and the area

CZMODE

CZMODE op[,cm[,am]]

Sets the operation, color and area modes that all drawing uses from then on. The values are in section 2.5. Modes left off are 0.

Argument	Meaning
op	Operation mode, 0-7 (0 replace, 1 OR, 2 AND, 3 XOR, 4-7 conditional)
cm	Color mode, 0-3
am	Area mode, 0-7

Example

```
CZMODE 3                ' XOR
CZLINE 0,0,500,300      ' draw it...
CZLINE 0,0,500,300      ' ...and XOR again: it is gone
CZMODE 0,0,2            ' replace, clipped to the area
```

CZAREA

CZAREA x1,y1,x2,y2

CZAREA

Sets the area rectangle the area modes test, with corners x1,y1 (bottom left) and x2,y2 (top right), both inside it. CZAREA with no arguments makes the area the whole screen again. The area only matters while CZMODE has an area mode set.

Errors: Syntax error with 1, 2 or 3 arguments.

Example

```
CZAREA 100,100,400,300
CZMODE 0,0,2            ' draw only inside it
CZCIRCLE 250,200,180     ' a clipped circle
CZMODE 0:CZAREA
```

3.4 The pen

CZMOVE

CZMOVE x,y

Moves the pen to x,y without drawing.

Pen afterwards: x,y

CZRMOVE

```
CZRMOVE dx,dy
```

Moves the pen by dx across and dy up, without drawing.

Pen afterwards: moved by dx,dy

3.5 Dots and lines

CZDOT

```
CZDOT x,y[,c]
```

Sets one pixel.

Argument	Meaning
x, y	The pixel
c	Color for this dot (default: the drawing color)

Pen afterwards: x,y

Example

```
FOR X=0 TO CZXMAX STEP 4:CZDOT X,CZYMAX\2:NEXT
```

CZLINE

```
CZLINE x1,y1,x2,y2[,c]
CZLINE TO x,y[,c]
```

Draws a straight line from x1,y1 to x2,y2, both ends included. CZLINE TO draws from the pen to x,y instead, so a chain of CZLINE TO statements draws a connected path.

Argument	Meaning
x1,y1 / x2,y2	The two ends
c	Color for this line

Pen afterwards: the end point (x2,y2 or x,y)

Lines follow the line style (CZLSTYLE) and the drawing mode. In XOR mode a chain of CZLINE TO lines draws each shared corner twice, which XORs it back out.

Example

```
CZMOVE 100,100
CZLINE TO 300,100:CZLINE TO 200,273:CZLINE TO 100,100   ' a triangle
```

CZRLINE

```
CZRLINE dx,dy[,c]
```

Draws a line from the pen to the point dx across and dy up from it, both ends included.

Pen afterwards: the end point

Example

```
CZMOVE 10,10:FOR I=1 TO 20:CZRLINE 20,0:CZRLINE 0,20:NEXT ' stairs
```

3.6 Rectangles

CZRECT

```
CZRECT x1,y1,x2,y2[,c]
```

Draws the outline of the rectangle with opposite corners x1,y1 and x2,y2 (any two opposite corners).

Pen afterwards: x1,y1

Example

```
CZRECT 0,0,CZXMAX,CZYMAX,255     ' frame the screen
```

CZFRECT

```
CZFRECT x1,y1,x2,y2[,c]
```

Fills the rectangle with opposite corners x1,y1 and x2,y2, in the current pattern (solid unless CZPAT or CZLSTYLE changed it).

Pen afterwards: just past the edge at y2 (the ACRTC's fill leaves it there); use CZMOVE before relying on it

Example

```
FOR I=0 TO 7:CZFRECT 40+I*120,60,140+I*120,260,I*32+31:NEXT
```

3.7 Circles, ellipses and arcs

CZCIRCLE

```
CZCIRCLE x,y,r[,c]
```

Draws a circle of radius r centred on x,y.

Argument	Meaning
x, y	The centre
r	The radius in pixels

Pen afterwards: the centre, x,y

Example

```
FOR R=20 TO 360 STEP 20:CZCIRCLE 512,384,R,64+R\12:NEXT
```

CZELLIPSE

CZELLIPSE x,y,rx,ry[,c]

Draws an ellipse centred on x,y with semi-axes rx across and ry up (the radius along X and along Y).

Argument	Meaning
rx, ry	The semi-axes, 0-4095

Pen afterwards: the centre, x,y

Example

```
CZELLIPSE 512,384,200,80,160+31
```

CZARC

CZARC xc,yc,xs,ys,xe,ye[,c[,dir]]

Draws part of a circle centred on xc,yc. It starts at xs,ys, so the radius is the distance from the centre to xs,ys, and runs round to where the circle meets the ray from the centre through xe,ye.

Argument	Meaning
xc, yc	The centre
xs, ys	The start point, on the circle
xe, ye	Any point on the ray where the arc ends
dir	0 (default): counter-clockwise; any other value: clockwise

Pen afterwards: xe,ye

Example

```
CZARC 300,300,400,300,300,400      ' a quarter circle, counter-clockwise
```

CZEARC

CZEARC xc,yc,rx,ry,xs,ys,xe,ye[,c[,dir]]

As CZARC, but on an ellipse centred on xc,yc whose semi-axes are in the ratio rx:ry. Only the ratio matters: the size comes from the start point xs,ys.

Argument	Meaning
rx, ry	The ratio of the semi-axes, each 0-90
dir	0: counter-clockwise; other: clockwise

Pen afterwards: xe,ye

Errors: Illegal function call if rx or ry is over 90.

Example

```
CZEARC 800,600,2,1,880,600,800,640,255     ' a quarter of a wide ellipse
```

3.8 Polygons and polylines

CZPOLY

```
CZPOLY a%(i),n[,c]
```

Draws a closed polygon through n points held in an integer array: element i is the first point's X, i+1 its Y, i+2 the second point's X, and so on. The last point is joined back to the first.

Argument	Meaning
a%(i)	The first element of the points, in an integer (%) array; 2n elements from there are used
n	The number of points, at least 2

Pen afterwards: the first point

Errors: Type mismatch if the array is not an integer array; Illegal function call if n is less than 2.

Example

```
10 DIM P%(7)
20 FOR I=0 TO 7:READ P%(I):NEXT
30 DATA 300,500,400,500,350,600,300,500
40 CZPOLY P%(0),4,64+20
```

CZPLINE

```
CZPLINE a%(i),n[,c]
```

As CZPOLY, but open: the last point is not joined back to the first.

Pen afterwards: the last point

Example

```
CZPLINE P%(0),3,255
```

3.9 Filling areas

CZPAINT

```
CZPAINT x,y,edge[,c]
```

Fills outward from x,y until it reaches pixels of color edge, in color c or the drawing color and the current pattern. The outline must be closed: the fill spreads up, down, left and right, so an outline of lines or a circle holds it.

Argument	Meaning
x, y	A point inside the area
edge	The color of the outline
c	The fill color

The fill also stops at pixels already in the fill color.

On the real board the ACRTC can fill only a limited number of separate spans at once, so a very complicated area may be left partly unfilled; fill it in pieces if it is.

Example

```
CZCIRCLE 400,400,50,96+31
CZPAINT 400,400,96+31,128+20    ' fill the circle red
```

CZFLOOD

`CZFLOOD x,y,region[,c]`

Recolors the connected region of color region that contains x,y: every other color is its edge. Use it to change the color of one shape.

Argument	Meaning
region	The color of the region to fill
c	Its new color

Example

```
CZFLOOD 400,400,128+20,192+31   ' the red circle turns yellow
```

3.10 Copying

CZCOPY

`CZCOPY x1,y1,x2,y2,xd,yd[,t]`

Copies the rectangle with corners x1,y1 and x2,y2 so that its corner x1,y1 lands on xd,yd, turned or mirrored as t says. The operation and area modes apply where it lands (so XOR and clipping work); the color mode does not, since the pixels bring their own colors.

Argument	Meaning
t	The transform, 0-7: 0 as it is, 1 flipped top to bottom, 2 mirrored left to right, 3 turned 180 degrees, 4 transposed (mirrored in the diagonal), 5 turned 90 degrees clockwise, 6 turned 90 degrees counter-clockwise, 7 mirrored in the other diagonal

Pen afterwards: moved; use CZMOVE before relying on it

Example

```
CZCOPY 100,100,200,200,250,100        ' a copy beside it
CZCOPY 100,100,200,200,450,100,2      ' mirrored
```

3.11 Patterns

CZPAT

CZPAT start,w0[,w1...]

Loads up to 16 words into the pattern RAM, from word start on. Each word is one row of the pattern (bit 0 the leftmost pixel). Words are 16-bit numbers: &H constants are handy, and -1 is all ones.

Argument	Meaning
start	The first word to load, 0-15
w0, w1, ...	The words (up to 16)

Example

```
CZPAT 0,&HAAAA,&H5555          ' a checkerboard, two rows
CZPSCAN 0,0,15,1               ' repeat just those rows
CZFRECT 20,300,80,360          ' a shaded box
```

CZLSTYLE

CZLSTYLE pat[,zoom]

Sets a line style: the 16 bits of pat, bit 0 first, repeat along every line, the 1 bits drawn and the 0 bits drawn in the background color (or skipped in color mode 1). Each bit covers zoom+1 pixels. CZLSTYLE -1 is solid again.

Argument	Meaning
pat	The style, 16 bits
zoom	0-15 (default 0): each bit drawn zoom+1 times

CZLSTYLE loads pattern row 0 and a 16 x 1 scan window, so fills use the style too until CZLSTYLE -1.

Example

```
CZLSTYLE &HF0F0             ' dashes
CZLINE 20,400,300,400
CZLSTYLE &HF0F0,2           ' longer dashes
CZLSTYLE -1                 ' solid
```

CZPSCAN

CZPSCAN x0,y0,x1,y1[,zx,zy]

Sets the scan window, the part of the pattern RAM the pattern pointer walks: columns x0 to x1 and rows y0 to y1. Each bit is drawn zx+1 times across and zy+1 times up. Also puts the pointer at x0,y0.

Argument	Meaning
x0, y0, x1, y1	The window, 0-15 each
zx, zy	The zoom, 0-15 each (default 0)

Example

```
CZPSCAN 0,0,15,15            ' all of it again
```

CZPTN

CZPTN x,y,w,h[,dir]

Draws the pattern RAM itself as a picture w pixels wide and h high with its corner at x,y, from the pattern pointer. It can be turned in 45-degree steps; adding 8 to dir slants the rows 45 degrees instead of 90. A diagonal pattern has gaps between its pixels, as the chip draws it.

Argument	Meaning
w, h	The size, 1-256
dir	0-7: the rows run dir x 45 degrees counter-clockwise from +X (default 0); +8 slants them

Pen afterwards: one row past the last row drawn

Example

```
CZPAT 0,&H0FF0,&H1FF8,&H3FFC,&H7FFE
CZPSCAN 0,0,15,3             ' those 4 rows, pointer at 0,0
CZPTN 900,700,16,4
CZPAT 0,-1,-1,-1,-1:CZPSCAN 0,0,15,15   ' solid again
```

3.12 Text

CZTEXT

CZTEXT x,y,s$[,c[,scale[,dir]]]

Draws the string s$ in CZBASIC's 8 x 8 font with the bottom-left corner of its first character at x,y. Only the characters' own pixels are drawn: the background shows through. Each character is scale times bigger than 8 x 8 pixels, and the text can run in any of eight directions.

Argument	Meaning
s$	The text: characters 32-126 (others draw as spaces)
c	Color for this text
scale	1-16 (default 1): each pixel of the font becomes a scale x scale block
dir	0-7 (default 0): the text runs dir x 45 degrees counter-clockwise from +X, the characters turned with it

Each character takes 8 x scale pixels along the line; the font's letters are 5 pixels wide and 7 high, with room for descenders and the gap to the next character.

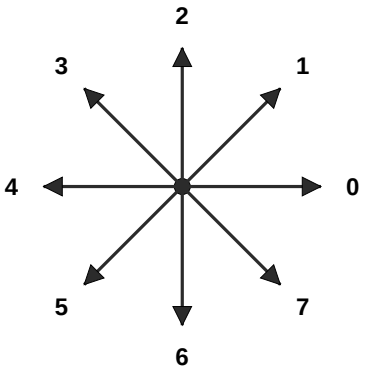
CZTEXT draws in the current operation and area modes. It uses the pattern RAM and leaves it solid, so a pattern or line style must be set again afterwards.

Errors: Illegal function call if scale is 0 or more than 16.

Example

```
CZTEXT 20,720,"CZBASIC ON CADZILLA",255,2
FOR D=0 TO 7:CZTEXT 512,384,"  DIRECTION",96+31,2,D:NEXT
```

Text directions (dir):



The CZTEXT character set (characters 20H-7EH):

20	21	22	23	24	25	26	27	28	29	2A	2B	2C	2D	2E	2F
30	31	32	33	34	35	36	37	38	39	3A	3B	3C	3D	3E	3F
40	41	42	43	44	45	46	47	48	49	4A	4B	4C	4D	4E	4F
50	51	52	53	54	55	56	57	58	59	5A	5B	5C	5D	5E	5F
60	61	62	63	64	65	66	67	68	69	6A	6B	6C	6D	6E	6F
70	71	72	73	74	75	76	77	78	79	7A	7B	7C	7D	7E	

4. Functions

CZPOINT

CZPOINT(x,y)

Returns the color of the pixel at x,y, 0-255, or -1 if x,y is off the screen. It reads the screen as it is after all the drawing before it.

Example

```
IF CZPOINT(X,Y)=0 THEN CZDOT X,Y,255   ' only on black
```

CZXMAX, CZYMAX

CZXMAX

CZYMAX

The largest X and Y of the screen in the current mode (1023 and 767 after CZINIT 1024). They take no arguments.

Example

```
CZRECT 0,0,CZXMAX,CZYMAX
```

CZXPOS, CZYPOS

CZXPOS

CZYPOS

The pen's current X and Y. After an area stop mode has stopped a drawing, they give the pixel where it stopped.

Example

```
CZCIRCLE 300,200,50:PRINT CZXPOS;CZYPOS     ' 300 200
```

5. Example programs

5.1 A star burst

```
10 CZINIT:CX=CZXMAX\2:CY=CZYMAX\2
20 FOR I=0 TO 63:A=I*6.28318/64
30   CZLINE CX,CY,CX+300*COS(A),CY+300*SIN(A),(I MOD 8)*32+31
40 NEXT
50 CZWAIT
```

5.2 A rubber band with XOR

Drawing in XOR mode twice leaves the screen as it was, so a line can be moved over a picture without damaging it:

```
10 CZINIT:CZCIRCLE 512,384,200,64+31:CZMODE 3
20 FOR X=100 TO 900 STEP 4
30   CZLINE 512,100,X,600,255    ' draw it
40   CZLINE 512,100,X,600,255    ' and take it away again
50 NEXT
60 CZMODE 0:CZWAIT
```

5.3 Shapes, paint and flood

```
10 CZINIT
20 DIM P%(11):FOR I=0 TO 5:A=I*6.28318/6
30   P%(2*I)=512+150*COS(A):P%(2*I+1)=384+150*SIN(A):NEXT
40 CZPOLY P%(0),6,255              ' a white hexagon
50 CZPAINT 512,384,255,128+20      ' filled red
60 CZFLOOD 512,384,128+20,32+24    ' then recolored blue
70 CZWAIT
```

5.4 Reading the screen

```
10 CZINIT:CZFRECT 200,200,300,300,96+31
20 N=0:FOR X=190 TO 310:IF CZPOINT(X,250)<>0 THEN N=N+1
30 NEXT:PRINT N;"pixels are cyan"       ' 101
```

5.5 Palette animation

Changing a palette entry recolors everything drawn in it at once, without redrawing:

```
10 CZINIT:FOR R=10 TO 350 STEP 10:CZCIRCLE 512,384,R,R\10:NEXT
20 FOR T=1 TO 100:FOR I=1 TO 35
30   CZPAL I,(I+T)*7 AND 255,0,255-((I+T)*7 AND 255)
40 NEXT:NEXT:CZPAL
```

Appendix A. Quick reference

Statement / function	Does
`CZINIT [640|800|1024]`	set up the board, clear, defaults
`CZCLS [c]`	clear the screen to c
`CZWAIT`	wait for the drawing to finish
`CZCOLOR fg[,bg]`	drawing and background colors
`CZPAL i,r,g,b | CZPAL`	set a palette entry | default palette
`CZMODE op[,cm[,am]]`	operation, color and area modes
`CZAREA x1,y1,x2,y2 | CZAREA`	the area | the whole screen
`CZMOVE x,y / CZRMOVE dx,dy`	move the pen
`CZDOT x,y[,c]`	a pixel
`CZLINE x1,y1,x2,y2[,c]`	a line, both ends drawn
`CZLINE TO x,y[,c]`	a line from the pen
`CZRLINE dx,dy[,c]`	a line relative to the pen
`CZRECT x1,y1,x2,y2[,c]`	rectangle outline
`CZFRECT x1,y1,x2,y2[,c]`	filled rectangle
`CZCIRCLE x,y,r[,c]`	circle
`CZELLIPSE x,y,rx,ry[,c]`	ellipse
`CZARC xc,yc,xs,ys,xe,ye[,c[,dir]]`	arc of a circle
`CZEARC xc,yc,rx,ry,xs,ys,xe,ye[,c[,dir]]`	arc of an ellipse
`CZPOLY a%(i),n[,c]`	closed polygon from an array
`CZPLINE a%(i),n[,c]`	open polyline from an array
`CZPAINT x,y,edge[,c]`	fill out to an edge color
`CZFLOOD x,y,region[,c]`	recolor a region
`CZCOPY x1,y1,x2,y2,xd,yd[,t]`	copy a block, turned or mirrored
`CZPAT start,w0[,w1...]`	load the pattern RAM
`CZLSTYLE pat[,zoom]`	line style
`CZPSCAN x0,y0,x1,y1[,zx,zy]`	pattern scan window and zoom
`CZPTN x,y,w,h[,dir]`	draw the pattern RAM
`CZTEXT x,y,s$[,c[,scale[,dir]]]`	text in the 8 x 8 font
`CZPOINT(x,y)`	a pixel's color, -1 off the screen
`CZXMAX CZYMAX`	the largest X and Y
`CZXPOS CZYPOS`	the pen position

Colors: index = hue + level; hues 0 orange, 32 blue, 64 green, 96 cyan, 128 red, 160 magenta, 192 yellow, 224 white; levels 0-31.
Coordinates: 0,0 at the bottom left, Y up.